

Chapter Five:

Network Programming

Introduction

- *network programming* refers to writing programs that execute across multiple devices (computers), in which the **devices are all connected to each other using a network**.
- The **java.net** package of the J2SE APIs contains a collection of classes and interfaces that provide the low-level communication details
- The java.net package provides support for the two common network protocols:
 - **TCP:** allows for **reliable communication** between two applications.
 - is typically used over the Internet Protocol, which is referred to as TCP/IP.
 - **UDP:** is a **connection-less** protocol that allows for packets of data to be transmitted between applications.

Socket Programming

- Sockets provide the communication mechanism between two computers using TCP.
- A client program creates a socket on its end of the communication and attempts to connect that socket to a server.
- When the connection is made, the server creates a socket object on its end of the communication.
- The client and server can now communicate by writing to and reading from the socket.
- The **java.net.Socket** class represents a socket, and
- The **java.net.ServerSocket** class provides a mechanism for the server program to listen for clients and establish connections with them.

- The following steps occur when establishing a TCP connection between two computers using sockets:
 - The server instantiates a `ServerSocket` object, denoting which port number communication is to occur on.
 - The server invokes the **`accept()`** method of the `ServerSocket` class. This method waits until a client connects to the server on the given port.
 - After the server is waiting, a client instantiates a `Socket` object, specifying **the server name and port number** to connect to.
 - The constructor of the `Socket` class attempts to connect the client to the specified server and port number.
 - If communication is established, the client now has a `Socket` object capable of communicating with the server.
 - On the server side, the `accept()` method returns a reference to a new socket on the server that is connected to the client's socket.

- After the connections are established, communication can occur using I/O streams.
- Each socket has both an OutputStream and an InputStream.
- The client's OutputStream is connected to the server's InputStream, and
- the client's InputStream is connected to the server's OutputStream.
- TCP is a two way communication protocol, so data can be sent across both streams at the same time.

ServerSocket Class

- The **java.net.ServerSocket** class is used by server applications to obtain a port and listen for client requests.
- **Constructors**
- All the four constructors throws IOException.
 - **ServerSocket(int port)**: Attempts to create a server socket bound to the specified port.
 - **ServerSocket(int port, int backlog)**: Similar to the previous constructor, the backlog parameter specifies **how many incoming clients to store in a wait queue**.
 - **ServerSocket(int port, int backlog, InetAddress address)**: the InetAddress parameter specifies the local IP address to bind to.
 - The **InetAddress** is used **for servers that may have multiple IP** addresses, allowing the server to specify which of its IP addresses to accept client requests on.

- **ServerSocket():** Creates an unbound server socket.
 - using this constructor, use the `bind()` method when you are ready to bind the server socket.
- Common Methods
 - **int getLocalPort():** Returns the port that the server socket is listening on.
 - This method is useful if you passed in 0 as the port number in a constructor and let the server find a port for you.
 - **Socket accept():** Waits for an incoming client.
 - This method blocks until either a client connects to the server on the specified port or the socket times out, assuming that the timeout value has been set using the `setSoTimeout()` method.
 - Otherwise, this method blocks indefinitely

- **void setSoTimeout(int timeout):** Sets the time-out value for how long the server socket waits for a client during the accept().
- **public void bind(SocketAddress host, int backlog):** Binds the socket to the specified server and port in the SocketAddress object.
 - Use this method if you instantiated the ServerSocket using the no-argument constructor.
- After a client does connect, the ServerSocket creates a new Socket on an unspecified port and returns a reference to this new Socket.
- A TCP connection now exists between the client and server, and communication can begin.

Socket Class

- The **java.net.Socket** class represents the socket that both the client and server use to communicate with each other.
- The client obtains a Socket object by instantiating one, whereas the server obtains a Socket object from the return value of the `accept()` method.
- The Socket class has five constructors that a client uses to connect to a server:
 - **Socket(String host, int port)**: attempts to connect to the specified server at the specified port.
 - throws **UnknownHostException, IOException**
 - **Socket(InetAddress host, int port)**: is identical to the previous constructor, except that the host is denoted by an InetAddress object.
 - throws **IOException**

- **Socket(String host, int port, InetAddress localAddress, int localPort):** Connects to the specified host and port, creating a socket on the local host at the specified address and port.
 - **throws IOException**
- **Socket(InetAddress host, int port, InetAddress localAddress, int localPort):** is identical to the previous constructor, except that the host is denoted by an InetAddress object instead of a String
 - **throws IOException**
- **Socket():** Creates an unconnected socket.
 - Use the connect() method to connect this socket to a server.

- Notice that both the client and server have a `Socket` object, so these methods can be invoked by both the client and server.
- Some methods of interest in the `Socket` class are:
 - **`void connect(SocketAddress host, int timeout)` throws `IOException`:** connects the socket to the specified host.
 - is needed only when you instantiated the `Socket` using the no-argument constructor.
 - **`InetAddress getAddress()`:** returns the address of the other computer that this socket is connected to.
 - **`int getPort()`:** Returns the port the socket is bound to on the remote machine.
 - **`int getLocalPort()`:** Returns the port the socket is bound to on the local machine.
 - **`SocketAddress getRemoteSocketAddress()`:** Returns the address of the remote socket.

- **InputStream getInputStream() throws IOException:** Returns the input stream of the socket.
 - The input stream is connected to the output stream of the remote socket.
- **OutputStream getOutputStream() throws IOException:** Returns the output stream of the socket.
 - The output stream is connected to the input stream of the remote socket
- **void close() throws IOException:** Closes the socket, which makes this Socket object no longer capable of connecting again to any server

InetAddress Class

- represents an Internet Protocol (IP) address
- useful methods which you would need while doing socket programming:
 - **InetAddress getByAddress(byte[] addr):** Returns an InetAddress object given the raw IP address.
 - **InetAddress getByAddress(String host, byte[] addr):** Create an InetAddress based on the provided host name and IP address.
 - **InetAddress getName(String host):** Determines the IP address of a host, given the host's name.
 - **String getHostAddress():** Returns the IP address string in textual presentation.
 - **String getHostName()** Gets the host name for this IP address.
 - **InetAddress getLocalHost():** Returns the local host.
 - **String toString():** Converts this IP address to a String.

Socket Client Example:

- The following **GreetingClient** is a client program that connects to a server by using a socket and sends a greeting, and then waits for a response.

```
import java.net.*;
import java.io.*;
public class GreetingClient {
    public static void main(String [] args) {
        String serverName = args[0];
        int port = Integer.parseInt(args[1]);
        try {
            System.out.println("Connecting to " +
                               serverName + " on port " + port);
            Socket client = new Socket(serverName, port);
```

```

        System.out.println("Just connected to " +
            client.getRemoteSocketAddress());
        OutputStream outToServer =
            client.getOutputStream();
        DataOutputStream out = new
            DataOutputStream(outToServer);
        out.writeUTF("Hello from " +
            client.getLocalSocketAddress());
        InputStream inFromServer =
            client.getInputStream();
        DataInputStream in = new
            DataInputStream(inFromServer);
        System.out.println("Server says " +
            in.readUTF());
        client.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Socket Server Example:

- The following GreetingServer program is an example of a server application that uses the Socket class to listen for clients on a port number specified by a command-line argument:

```
import java.net.*;
import java.io.*;
public class GreetingServer extends Thread {
    private ServerSocket serverSocket;
    public GreetingServer(int port) throws IOException {
        serverSocket = new ServerSocket(port);
        serverSocket.setSoTimeout(10000);
    }
```



```

public void run() {
    while(true) {
        try {
            System.out.println("Waiting for client on port " +
                serverSocket.getLocalPort() + "...");
            Socket server = serverSocket.accept();
            System.out.println("Just connected to " +
                server.getRemoteSocketAddress());
            DataInputStream in = new
                DataInputStream(server.getInputStream());
            System.out.println(in.readUTF());
            DataOutputStream out = new
                DataOutputStream(server.getOutputStream());
            out.writeUTF("Thank you for connecting to " +
                server.getLocalSocketAddress() + "\nGoodbye!");
            server.close();
        }
    }
}

```

```

        catch(SocketTimeoutException s) {
            System.out.println("Socket timed out!");
            break;
        }catch(IOException e) {
            e.printStackTrace();
            break;
        }
    }
}

public static void main(String [] args) {
    int port = Integer.parseInt(args[0]);
    try {
        Thread t = new GreetingServer(port);
        t.start();
    }catch(IOException e) {
        e.printStackTrace();
    }
}
}

```

- start server as follows:
 - `$ java GreetingServer 6000`
- Check client program as follows:
 - `$ java GreetingClient localhost 6000`

URL

- The Web is a loose collection of higher-level protocols and file formats, all unified in a web browser.
- Once you can reliably name anything and everything, it becomes a very powerful paradigm. The Uniform Resource Locator (URL) does exactly that.
- The URL provides a reasonably intelligible form to uniquely identify or address information on the Internet.
- URLs are ubiquitous; every browser uses them to identify information on the Web.
- Within Java's network class library, the URL class provides a simple, concise API to access information across the Internet using URLs.

- **Format**

- A URL specification is based on **four components**:
 - The first is the **protocol** to use, separated from the rest of the locator by a colon (:).
 - The second component is the **host name** or **IP address** of the host to use; this is delimited on the left by double slashes (//) and on the right by a slash (/) or optionally a colon (:).
 - The third component, the **port number**, is an optional parameter, delimited on the left from the host name by a colon (:) and on the right by a slash (/).
 - The fourth part is the **actual file path**.
- Java's URL class has several constructors, and each can throw a **MalformedURLException**.
- One commonly used form specifies the URL with a string that is identical to what you see displayed in a browser:
 - `URL(String urlSpecifier)`

- The next two forms of the constructor allow you to break up the URL into its component parts:
 - `URL(String protocolName, String hostName, int port, String path)`
 - `URL(String protocolName, String hostName, String path)`
- Another frequently used constructor allows you to use an existing URL as a reference context and then create a new URL from that context.
 - `URL(URL urlObj, String urlSpecifier)`

- **Commonly used methods of Java URL class**

- The java.net.URL class provides many methods. The important methods of URL class are given below.

<u>Method</u>	<u>Description</u>
– <code>public String getProtocol()</code>	it returns the protocol of the URL.
– <code>public String getHost()</code>	it returns the host name of the URL.
– <code>public String getPort()</code>	it returns the Port Number of the URL.
– <code>public String getFile()</code>	it returns the file name of the URL.
– <code>public URLConnection openConnection() this URL.</code>	it returns the instance of URLConnection i.e associated with

- In the following example, we create a URL to SW web page and then examine its properties:

```
import java.net.*;

class URLDemo {
    public static void main(String args[]) throws
        MalformedURLException {
        URL hp = new URL("
            http://sw.aastu.edu.et/index.php/about-sw/");
        System.out.println("Protocol: " + hp.getProtocol());
        System.out.println("Port: " + hp.getPort());
        System.out.println("Host: " + hp.getHost());
        System.out.println("File: " + hp.getFile());
        System.out.println("Ext:" + hp.toExternalForm());
    }
}
```


- output:
 - Protocol: http
 - Port: -1
 - Host: sw.aastu.edu.et
 - File: /index.php/about-sw/
 - Ext:<http://sw.aastu.edu.et/index.php/about-sw/>
- To access the actual bits or content information of a URL, you create a **URLConnection** object from it, using its **openConnection()** method, like this:
 - url.openConnection()
- openConnection() has the following general form:
 - URLConnection openConnection()
- It returns a URLConnection object associated with the invoking URL object. It may throw an **IOException**.

URLConnection Class

- represents a communication link between the URL and the application.
- This class can be used to read and write data to the specified resource referred by the URL.
- The `openConnection()` method of URL class returns the object of `URLConnection` class.
- Syntax:

```
public URLConnection openConnection()throws IOException{}
```
- The `URLConnection` class provides many methods, **we can display all the data of a webpage** by using the **`getInputStream()`** method.
- The `getInputStream()` method returns all the data of the specified URL in the stream that can be read and displayed.

Example of Java URLConnecton class

```
import java.io.*;
import java.net.*;
public class URLConnectionExample {
    public static void main(String[] args){
        try{
            URL url=new URL("http://www.javatpoint.com/java-tutorial");
            URLConnection urlcon=url.openConnection();
            InputStream stream=urlcon.getInputStream();
            int i;
            while((i=stream.read())!=-1){
                System.out.print((char)i);
            }
        }catch(Exception e){System.out.println(e);}
    }
}
```

HttpURLConnection class

- is http specific URLConnection. It works for HTTP protocol only.
- By the help of HttpURLConnection class, you can get information of any HTTP URL such as header information, status code, response code etc.
- The `java.net.HttpURLConnection` is subclass of `URLConnection` class.
- **How to get the object of HttpURLConnection class**
 - Get the `URLConnection` object using `openConnection()` method
 - `URLConnection openConnection() {}`
 - Typecast this object to `HttpURLConnection` type
 - `URL url=new URL("http://www.javatpoint.com/java-tutorial");`
 - `HttpURLConnection huc=(HttpURLConnection)url.openConnection();`

URLConnection Example

```
import java.io.*;
import java.net.*;

public class HttpURLConnectionDemo {
    public static void main(String[] args) {
        try {
            URL url=new URL("http://sw.aastu.edu.et/");
            HttpURLConnection huc=(HttpURLConnection)url.openConnection();
            for(int i=1;i<=8;i++){
                System.out.println (huc.getHeaderFieldKey(i) + " = " +
                huc.getHeaderField(i));
            }
            huc.disconnect();
        } catch(Exception e) {System.out.println(e);}
    }
}
```

DatagramSocket and DatagramPacket

- Java `DatagramSocket` and `DatagramPacket` classes are used for connection-less socket programming.
- **DatagramSocket class**
 - represents a connection-less socket for sending and receiving datagram packets.
 - A datagram is basically an information but there is no guarantee of its content, arrival or arrival time.
 - **Commonly used Constructors of DatagramSocket class**
 - **DatagramSocket() throws SocketException:** it creates a datagram socket and binds it with the available Port Number on the localhost machine.
 - **DatagramSocket(int port) throws SocketException:** it creates a datagram socket and binds it with the given Port Number.
 - **DatagramSocket(int port, InetAddress address) throws SocketException:** it creates a datagram socket and binds it with the specified port number and host address.

- **DatagramPacket class**

- is a message that can be sent or received.
- If you send multiple packet, it may arrive in any order. Additionally, packet delivery is not guaranteed.
- **Commonly used Constructors of DatagramPacket class**
 - **DatagramPacket(byte[] barr, int length):** it creates a datagram packet. This constructor is used to receive the packets.
 - **DatagramPacket(byte[] barr, int length, InetAddress address, int port):** it creates a datagram packet. This constructor is used to send the packets.

Example of Sending DatagramPacket by DatagramSocket

```
import java.net.*;

public class DSender{
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket();
        String str = "Welcome java";
        InetAddress ip = InetAddress.getByName("127.0.0.1");
        DatagramPacket dp = new DatagramPacket(str.getBytes(),
            str.length(), ip, 3000);
        ds.send(dp);
        ds.close();
    }
}
```


Example of Receiving DatagramPacket by DatagramSocket

```
import java.net.*;

public class DReceiver{
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket(3000);
        byte[] buf = new byte[1024];
        DatagramPacket dp = new DatagramPacket(buf, 1024);
        ds.receive(dp);
        String str = new String(dp.getData(), 0, dp.getLength());
        System.out.println(str);
        ds.close();
    }
}
```

